

Introdução à Lógica e à Programação

Listas: parte II

Fatiamento (slicing)

É um recurso que permite usar índices para copiar uma lista inteira ou uma sublista. O fatiamento é usado com o operador de dois pontos, o qual recebe o **índice inicial** (incluído) e o **índice final** (excluído) do intervalo cujos elementos serão copiados.

```
a = [10, 20, 30, 40, 50, 60, 70]
b = a[2:5]
print(b) # [30, 40, 50]
# omitir o primeiro índice significa índice zero
print(a[0:3]) # [10, 20, 30]
print(a[:3]) # [10, 20, 30]
# omitir o segundo índice significa índice final
print(a[5:7]) # [60, 70]
print(a[5:]) # [60, 70]
# copiando a lista inteira
b = a[:]
print(b) # [10, 20, 30, 40, 50, 60, 70]
```

10	20	30	40	50	60	70
0	1	2	3	4	5	6

Fatiamento (slicing)

O recurso de fatiamento trabalha com **passo** (step), cujo valor padrão é igual a 1. Note que o índice final pode ser menor do que o índice inicial caso o passo seja negativo.

```
a = [10, 20, 30, 40, 50, 60, 70]
# passo 2
print(a[0:5:2]) # [10, 30, 50]
print(a[::2])   # [10, 30, 50, 70]
# passo 3
print(a[0:4:3]) # [10, 40]
# a omissão do passo funciona como passo 1
print(a[1:3])   # [20, 30]
print(a[1:3:])  # [20, 30]
# passo negativo
print(a[4:0:-1]) # [50, 40, 30, 20]
print(a[6:1:-2]) # [70, 50, 30]
# sublista vazia
print(a[6:1])    # []
print(a[1:5:-2]) # []
```

10	20	30	40	50	60	70
0	1	2	3	4	5	6

Fatiamento (slicing)

Índices negativos também podem ser utilizados.

```
a = [10, 20, 30, 40, 50, 60, 70]
print(a[-6:-3])    # [20, 30, 40]
print(a[0:-1])     # [10, 20, 30, 40, 50, 60]
print(a[-7:-2:2])  # [10, 30, 50]
print(a[-3:-6])    # []
# revertendo a lista
print(a[::-1])     # [70, 60, 50, 40, 30, 20, 10]
```

-7	-6	-5	-4	-3	-2	-1
10	20	30	40	50	60	70
0	1	2	3	4	5	6

Exercício

1) Dada a lista $L=[5, 0, 3, 8, 9, 4, 1, 2]$, qual é a sublista resultante para cada um dos fatiamentos a seguir?

- a) $L[0:]$
- b) $L[:5]$
- c) $L[2:7]$
- d) $L[3:4]$
- e) $L[-7:-3]$
- f) $L[:15]$
- g) $L[::2]$
- h) $L[1::3]$
- i) $L[-2::-1]$

Copiando uma lista com fatiamento

Uma lista pode ser copiada usando o recurso de fatiamento. Lembre-se que a atribuição entre variáveis faz com que ambas variáveis operem sobre a mesma lista.

```
a = ['x', 'y', 'z']  
b = a  
c = a[:]  
b[0] = 'W' # a e b operam sobre a mesma lista  
print(a)   # ['W', 'y', 'z']  
print(b)   # ['W', 'y', 'z']  
print(c)   # ['x', 'y', 'z']
```

Comparação de listas

O operador `==` pode ser utilizado para comparar duas listas. O resultado é verdadeiro apenas quando as duas listas possuem a mesma sequência de elementos.

```
a = [5, 7, 9]
b = [2, 4, 6]
c = [5, 7, 9]
d = ['5', '7', '9']
e = [5, 9, 7]
f = [5, 7, 9, 1]
print(a == a) # True
print(a == b) # False
print(a == c) # True
print(a == d) # False
print(a == e) # False
print(a == f) # False
```

Função enumerate

Para cada elemento na lista, gera uma tupla com índice e elemento. É útil quando precisamos percorrer uma lista e ter acesso aos índices.

```
# sem enumerate
a = [10, 20, 30, 40]
for i in range(len(a)):
    print(f'a[{i}] = {a[i]}')
```

```
# com enumerate
a = [10, 20, 30, 40]
for i, e in enumerate(a):
    print(f'a[{i}] = {e}')
```

Exercício

2) Qual é a saída do programa a seguir?

```
L = [3, -6, 5, 0, 2]
for i, e in enumerate(L):
    print(f'{e} ** {i} = {e ** i}')
```

Método index

Retorna o índice da primeira ocorrência de determinado elemento.

```
a = ["a", "e", "b", "a"]  
print(a.index("b")) # 2  
print(a.index("a")) # 0  
print(a.index("f")) # gera erro
```

Método count

Retorna a quantidade de ocorrências de determinado elemento.

```
a = ["a", "e", "b", "a"]  
print(a.count("b")) # 1  
print(a.count("a")) # 2  
print(a.count("f")) # 0
```

Método extend

Adiciona, na lista em que o método é chamado, os elementos de uma outra lista.

```
a = ["a", "e", "b", "a"]
b = [35, 4, 9]
print(a) # ["a", "e", "b", "a"]
print(b) # [35, 4, 9]
a.extend(b)
print(a) # ["a", "e", "b", "a", 35, 4, 9]
print(b) # [35, 4, 9]
```

Remoção de elementos

Podemos utilizar um dos seguintes métodos para remover elementos de uma lista:

- `pop`: remove o elemento presente no índice indicado.
- `remove`: remove a primeira ocorrência do elemento indicado.
- `clear`: remove todos os elementos da lista.

```
a = ['a', 'e', 'b', 'a', 'z']  
a.pop(2)  
print(a) # ['a', 'e', 'a', 'z']  
a.remove("z")  
print(a) # ['a', 'e', 'a']  
a.remove("a")  
print(a) # ['e', 'a']  
a.clear()  
print(a) # []
```

Método reverse

Inverte a sequência de elementos da lista.

```
a = [1, 2, 3, 4]
print(a) # [1, 2, 3, 4]
a.reverse()
print(a) # [4, 3, 2, 1]
```

Método sort

Ordena a sequência de elementos da lista. O parâmetro `reverse` indica se a ordenação deve ser crescente (`False`) ou decrescente (`True`). O valor padrão é `False`.

```
a = [3, 5, 2, 1, 4]
a.sort()
print(a) # [1, 2, 3, 4, 5]
a.sort(reverse=True)
print(a) # [5, 4, 3, 2, 1]
a.sort(reverse=False)
print(a) # [1, 2, 3, 4, 5]
```

Exercício

3) Escreva uma função que tem uma lista como parâmetro e indica se essa lista é idêntica à sua reversão. Escreva pelo menos quatro testes da função no programa principal.

Exemplo de lista	Retorno esperado
[1, 2, 2, 1]	True
[]	True
[3]	True
[1, 2, 3]	False
[1, -1]	False

Exercício

4) Escreva uma função que tem uma lista como parâmetro e que remove os elementos ímpares dessa lista. Escreva pelo menos três testes da função no programa principal.

Exemplo de lista	Resultado esperado
[1, 2, 3, 4, 5, 6]	[2, 4, 6]
[3, 5, 9, 11]	[]
[10, 20, 30, 40, 50]	[10, 20, 30, 40, 50]

Exercício

5) Escreva uma função que tem como parâmetros duas listas **A** e **B** e que retorna uma nova lista contendo os elementos de **A** que não estão em **B**. Escreva pelo menos três testes da função no programa principal.

Exemplo de parâmetros	Retorno esperado
Lista A: [10, 20, 30, 40, 50] Lista B: [15, 20, 25, 30]	[10, 40, 50]
Lista A: [10, 20, 30, 40, 50] Lista B: [15, 25, 35]	[10, 20, 30, 40, 50]
Lista A: [1, 1, 1, 1, 2, 1, 1, 2] Lista B: [1]	[2, 2]
Lista A: [1, 2, 1, 1, 2] Lista B: [2, 1]	[]

Exercício

6) Escreva um programa que tem como entrada quatro números inteiros e os imprime em ordem crescente.

Exemplo de entrada	Saída esperada
5 0 1 -2	-2 0 1 5
1 3 1 2	1 1 2 3

Exercício

7) Escreva uma função que tem como parâmetros uma lista de strings **A** e uma lista de inteiros **B**, sendo que **B** representa a quantidade de ocorrências das palavras em **A** no respectivo índice. A função deve retornar uma cópia de **A** ordenada de forma decrescente com base nos valores presentes em **B**. Escreva pelo menos dois testes da função no programa principal.

Exemplo de parâmetros	Retorno esperado
Lista A: ['mamão', 'cajú', 'banana'] Lista B: [5, 9, 1]	['cajú', 'mamão', 'banana']
Lista A: ['bola', 'casa', 'rato', 'boné'] Lista B: [10, 20, 30, 40]	['boné', 'rato', 'casa', 'bola']

Referências

MENEZES, Nilo N. C.; **Introdução à Programação com Python**: Algoritmos e lógica de programação para iniciantes. 2ª ed. São Paulo: Novatec, 2014.

W3Schools. **Python Tutorial**. Disponível em
<<https://www.w3schools.com/python/default.asp>>. Acesso em 4/11/2024.